

Android 中.9 图的代码实现方案

在 Android 中，.9 图（9-Patch 图片）的核心功能是指定可拉伸区域和内容显示区域，这类效果完全可以通过代码实现，无需依赖提前制作的 .9.png 资源文件。以下是具体实现方案、原理和使用场景，帮你全面理解：

一、核心原理：.9 图的本质 vs 代码实现逻辑

.9 图的核心是两个关键区域，代码实现的本质就是通过 API 模拟这两个区域的规则：

.9 图的核心特性	代码中的对应实现方式
1. 可拉伸区域（黑色横/竖线标记）	通过 <code>setPadding()</code> 或 <code>inset</code> 控制“不可拉伸的边缘”，剩余中间区域自动拉伸
2. 内容显示区域（黑色内框标记）	通过 <code>setContentInsets()</code> 或 <code>padding</code> 限制子视图/文字的显示范围，避免内容覆盖关键区域

简单说：代码实现的核心是「通过“内边距（padding）”或“嵌入（inset）”分割图片的“固定边缘”和“可拉伸中间”，再配合图片的拉伸模式（如 `ScaleType.FIT_XY`）实现 .9 图效果」。

二、3 种常用代码实现方案（按场景选择）

方案 1：用 `ShapeDrawable` 实现纯色/渐变的 .9 图效果（推荐，无图片依赖）

如果你的 .9 图是**纯色背景、渐变背景**（比如按钮背景、输入框边框），直接用 `ShapeDrawable` 代码绘制，无需任何图片资源，灵活度最高。

示例：实现“上下固定高度、中间拉伸”的纯色按钮背景（类似 .9 图）

Code block

```
1 // 1. 创建 ShapeDrawable (纯色矩形, 对应 .9 图的“底图”)
2 val shapeDrawable = ShapeDrawable(RectShape()).apply {
3     // 设置背景色 (对应 .9 图的底色)
4     paint.color = ContextCompat.getColor(context, R.color.blue)
5
6     // 2. 模拟 .9 图的“可拉伸区域”: 通过 setPadding 固定上下边缘 (比如各 10dp)
```

```

7      // 注意：这里的 padding 是“不可拉伸的边缘宽度”，中间剩余区域会自动拉伸
8      val paddingVertical = dp2px(10) // 上下固定边缘: 10dp
9      val paddingHorizontal = dp2px(5) // 左右固定边缘: 5dp
10     setPadding(paddingHorizontal, paddingVertical, paddingHorizontal,
paddingVertical)
11
12     // 3. 模拟 .9 图的“内容显示区域”：限制文字/子视图的显示范围（可选，避免内容贴边）
13     // 比如让文字距离左右边缘 15dp，上下边缘 8dp
14     setContentInsets(dp2px(15), dp2px(8), dp2px(15), dp2px(8))
15 }
16
17 // 4. 给 View 设置背景（比如 Button、TextView）
18 button.background = shapeDrawable

```

辅助工具方法（dp 转 px，避免适配问题）

Code block

```

1 fun dp2px(dp: Int): Int {
2     return TypedValue.applyDimension(
3         TypedValue.COMPLEX_UNIT_DIP,
4         dp.toFloat(),
5         Resources.getSystem().displayMetrics
6     ).toInt()
7 }

```

效果说明：

- 按钮拉伸时，上下 10dp、左右 5dp 的边缘不会拉伸（类似 .9 图的黑色横线/竖线标记的不可拉伸区域）；
- 按钮内的文字只会在「左右 15dp、上下 8dp」范围内显示（类似 .9 图的黑色内框标记的内容区域）。

方案 2：用 `NinePatchDrawable` 加载普通图片，代码指定拉伸/内容区域

如果你的 .9 图是带图案的图片（比如带圆角的图片背景），可以用 `NinePatchDrawable` 直接包裹普通图片，通过代码指定 .9 图的核心规则，无需提前制作 .9.png 文件。

示例：将普通图片（icon_bg.png）转为代码版 .9 图

Code block

```

1 // 1. 加载普通图片资源（非 .9.png，就是普通 png/jpg）
2 val bitmap = BitmapFactory.decodeResource(resources, R.drawable.icon_bg)
3
4 // 2. 定义 .9 图的关键参数：

```

```

5 // - 拉伸区域 (left, top, right, bottom) : 对应 .9 图的黑色横/竖线位置
6 // 比如: 左边缘 20px 不可拉伸, 上边缘 15px 不可拉伸, 右边缘 20px 不可拉伸, 下边缘
   15px 不可拉伸
7 // 中间 (20px~bitmap.width-20px, 15px~bitmap.height-15px) 是可拉伸区域
8 val stretchLeft = 20
9 val stretchTop = 15
10 val stretchRight = bitmap.width - 20
11 val stretchBottom = bitmap.height - 15
12
13 // - 内容区域 (可选) : 对应 .9 图的黑色内框, 限制子视图显示范围
14 val contentLeft = 30
15 val contentTop = 20
16 val contentRight = bitmap.width - 30
17 val contentBottom = bitmap.height - 20
18
19 // 3. 创建 NinePatchDrawable (代码版 .9 图)
20 val ninePatchDrawable = NinePatchDrawable(
21     resources,
22     bitmap,
23     // 拉伸区域的标记 (本质是 .9 图的黑色像素点, 代码中直接指定坐标即可)
24     NinePatch.createStretchPatchingData(
25         bitmap,
26         stretchLeft,
27         stretchTop,
28         stretchRight,
29         stretchBottom
30     ),
31     // 内容区域的标记 (可选, 无则用拉伸区域)
32     Rect(contentLeft, contentTop, contentRight, contentBottom),
33     null // 密度参数, 默认 null 即可
34 )
35
36 // 4. 给 View 设置背景
37 textView.background = ninePatchDrawable

```

关键说明:

- `stretchLeft/Top/Right/Bottom` : 定义图片的「可拉伸区域」, 比如 `stretchLeft=20` 表示图片左边缘 20px 不可拉伸, 右侧同理;
- 普通图片会被自动处理为“中间拉伸、边缘固定”, 效果和 .9.png 完全一致;
- 优点: 无需修改图片资源, 直接用代码适配不同场景的拉伸规则。

方案 3: 用 `InsetDrawable` + `ScaleDrawable` 组合实现复杂 .9 效果

如果需要更复杂的逻辑（比如“图片+边框”“多层拉伸”），可以组合 `InsetDrawable`（控制边缘缩进）、`ScaleDrawable`（控制拉伸比例）、`LayerDrawable`（多层叠加）实现。

示例：实现“外层边框+内层拉伸图片”的复合 .9 效果

Code block

```
1  // 1. 内层：可拉伸的图片
2  val innerBitmap = BitmapFactory.decodeResource(resources, R.drawable.inner_bg)
3  val innerNinePatch = NinePatchDrawable(
4      resources,
5      innerBitmap,
6      NinePatch.createStretchPatchingData(innerBitmap, 10, 10, innerBitmap.width-
7      10, innerBitmap.height-10),
8      null,
9      null
10 )
11 // 2. 外层：边框（用 ShapeDrawable 绘制）
12 val borderShape = ShapeDrawable(RectShape()).apply {
13     paint.color = ContextCompat.getColor(context, R.color.gray)
14     paint.style = Paint.Style.STROKE // 边框模式
15     paint.strokeWidth = dp2px(2).toFloat() // 边框宽度 2dp
16 }
17
18 // 3. 组合内层和外层（LayerDrawable 实现叠加）
19 val layerDrawable = LayerDrawable(arrayOf(
20     innerNinePatch, // 下层：拉伸背景
21     borderShape      // 上层：边框
22 )).apply {
23     // 给边框设置内边距，避免覆盖内层图片
24     setLayerInset(1, dp2px(2), dp2px(2), dp2px(2), dp2px(2), dp2px(2))
25 }
26
27 // 4. 给 View 设置背景
28 cardView.background = layerDrawable
```

三、代码实现 vs 传统 .9.png 对比（怎么选？）

特性	代码实现	传统 .9.png 资源
灵活性	极高，可动态修改拉伸/内容区域	固定，修改需重新制作图片
适配性		

	自动适配所有分辨率（dp 转 px）	需为不同分辨率（xhdpi/xxhdpi）准备图片
资源体积	无图片资源，体积更小	需占用图片资源空间
复杂度	需写代码，有一定学习成本	可视化工具制作（Android Studio），简单直观
适用场景	纯色/渐变背景、动态规则（比如根据主题切换拉伸区域）	带复杂图案的背景、快速开发场景

四、注意事项

1. 图片拉伸模式：如果用普通图片实现，建议确保图片的“可拉伸区域”是纯色或重复图案，避免拉伸后变形；
2. 性能：代码绘制的 `ShapeDrawable` 性能优于图片资源（无需解码图片），推荐优先使用；
3. 版本兼容性：`NinePatchDrawable`、`ShapeDrawable` 是 Android 原生 API，支持所有版本（API 1+），无兼容性问题；
4. 工具辅助：如果想快速确定拉伸区域坐标，可以先用 Android Studio 的「9-Patch Drawable Editor」打开图片，可视化调整拉伸区域，再将坐标对应到代码中。

总结

.9 图的核心功能（拉伸区域+内容区域）完全可以通过代码实现，其中：

- 纯色/渐变背景：优先用 `ShapeDrawable`（简单、高效、无资源依赖）；
- 带图案背景：用 `NinePatchDrawable` 包裹普通图片，代码指定拉伸规则；
- 复杂效果：组合 `LayerDrawable` / `InsetDrawable` 实现多层叠加。

如果需要动态修改拉伸规则（比如根据用户设置调整边缘宽度），代码实现是唯一选择；如果是固定效果，传统 .9.png 制作更快捷。

（注：文档部分内容可能由 AI 生成）